Is TPC Still Not Enough?

Replicating the Amazon Redshift fleet study (VLDB 2024) on the public Redset trace, and measuring what an ideal result cache would save

Mohamed Abderrahmane Heouaine

October 2026

Abstract

van Renen et al. (PVLDB 17(11), 2024) argue, from internal Amazon Redshift telemetry, that the TPC-H and TPC-DS benchmarks no longer match what cloud data warehouses run: write-heavy pipelines, repetitive queries, bursty load and long-tailed distributions. With the paper, Amazon released *Redset*, a trace of queries from 200 provisioned and 200 serverless clusters over three months of 2024. This report asks whether the paper's conclusions hold on that public data. I rebuilt the paper's metrics as tested SQL over the full trace (433,107,731 provisioned and 8,244,381 serverless queries) on a single laptop with DuckDB, and compared 29 quantitative claims: 17 are confirmed, 7 partially confirmed, 5 not confirmed. The core argument survives: about two thirds of the runtime goes to writes, the runtime tail is even longer than reported, and in the median provisioned cluster 79 % of queries repeat (paper: 80 %). The query mix (fewer selects), the usage of CTAS tables and the number of distinct table sets differ markedly. Three properties of the trace had to be handled first; the most consequential is that result-cache hits are logged as `other`, which hides them from every select-based metric. As an extension, I replay every select against an ideal result cache with table-level invalidation: it would answer 63.1 % of provisioned selects instead of the 33.6 % Redshift answered, and save 25.1 % of select runtime instead of 8.3 %. The model classifies 97.5 % of the real cache hits as hits, which validates it.

1 Introduction

Benchmarks shape what database systems optimize. The paper replicated here [1] compares one month of telemetry from the provisioned Amazon Redshift fleet with TPC-H and TPC-DS and concludes that a modern benchmark must model writes, repetition, elasticity and long tails. Its numbers cannot be checked directly: the telemetry is private. The authors released a public dataset, Redset [2], "as a basis for building more realistic benchmarks". It is a different sample (other clusters, a later period, serverless included) with fewer columns (no query text, no filters, no memory, no table statistics).

This is therefore a *conceptual replication*: the same metrics, recomputed independently on different data from the same system. For each quantitative claim the question is whether it survives on the data anyone can download. The contributions are:

1. an open pipeline (DuckDB SQL, about one hour on a 16-thread laptop) that recomputes the paper's Sections 3–5 on the full Redset, with unit tests on hand-made traces;
2. a scorecard of 29 claims for the provisioned and serverless variants (Table 1);
3. three properties of Redset that change the results if ignored (Section 2.1);
4. an extension that the paper motivates but does not quantify: the hit rate and saved runtime of an ideal result cache, validated against Redshift's real cache, and its sensitivity to an eviction timeout (Section 4).

2 Data and method

Redset. One row per query: cluster, user and database ids, arrival time, compile, queue and execution times, query type, a hash of the query's features (`feature_fingerprint`), abort and result-cache flags, numbers and ids of the tables read and written, megabytes scanned and spilled, and the numbers of joins, scans and aggregations in the plan. Clusters were sampled to cover all levels of activity, so fleet-wide shares

are shares *of this sample*. After restricting to 1 March–30 May 2024 (the trace ends at midnight on 31 May; 81 rows fall outside), it holds 441.4 million queries.

Pipeline. The 17.7 GB of Parquet files are loaded once into a typed DuckDB table (fingerprints hashed to 64-bit integers, table-id strings parsed into lists). Each metric is a SQL query whose result is saved as a CSV; every number below is read from those files by a script, not typed by hand. TPC-H and TPC-DS run in DuckDB at scale factor 10 on the same laptop; absolute runtimes are therefore not comparable with the paper's 10-node cluster at SF 3000, only the shape of their distribution and the plan operators per query.

Metric definitions. Runtime is the execution time. *Spikiness* is described in the paper as "the sum of the root mean square errors between the current and previous day" without a normalization. I normalize each series to sum to one and sum the absolute differences of consecutive values (the RMSE of two scalars), over the days of a month (daily) or the 24 hours of the day (hourly). On the aggregated provisioned fleet this gives 0.11 daily and 0.11 hourly against the paper's 0.12 and 0.09, which supports this reading. *Repetition* follows the paper: (queries − distinct queries) / queries in a window, with the fingerprint standing in for the query. Redset's documentation warns that the fingerprint overestimates repetition, so it sits between the paper's "query" and "query template" metrics.

2.1 Three properties of the trace

1. **Result-cache hits are logged as `other`**, with no fingerprint and no tables, plus a pointer to the query that produced the result (a select in 99.8 % of cases). Left as is, the real cache hit rate among selects is 0 %, selects are undercounted by a third and repetition is underestimated. I restore them as selects with their source's fingerprint and tables (96.2 % of provisioned hits have their source in the trace).
2. **The trace ends on 30 May.** Keeping 31 May adds an almost empty day and a fake drop in load.
3. **Every CTAS creates a new table id** (25.36 M creations, 25.34 M distinct target ids). Re-creation of "the same table" is measured through the CTAS statement's fingerprint instead. The same effect on staging tables inflates the number of distinct table sets (Section 3.4).

3 Results

Table 1 lists every claim. The sections below discuss them in the paper's order.

Table 1: Replication scorecard. Paper values are from the paper's tables and text; Redset values are computed on the full trace. "Partial": same direction, clearly different size or a proxy metric.

§	Claim	Paper	Provisioned	Serverless	Verdict
3.2	Queries that are selects	48.9 %	30.5 %	30.9 %	not confirmed
3.2	Runtime spent in writes (insert, copy, delete, update, CTAS)	66.8 %	65.8 %	60.8 %	confirmed
3.3	Runtime spent in CTAS	16.4 %	26.5 %	3.6 %	partial
3.4	Queries under 100 ms	62.0 %	62.4 %	40.1 %	confirmed
3.4	Runtime in queries over 1 h	15.9 %	12.7 %	30.8 %	confirmed
4.3	Runtime tail: p99.9 / median	2,030x	7,483x	7,066x	confirmed
3.4	Scans per executed query	2.03	1.31	0.74	partial
3.4	Joins per executed query	0.53	0.52	0.56	confirmed
4.1	Fleet daily spikiness (March)	0.12	0.11	0.15	confirmed
4.2	Fleet hourly spikiness (March)	0.09	0.11	0.13	confirmed
4.1	Clusters at most as spiky as the paper's C2 (daily, 0.28)	75.0 %	50.0 %	12.5 %	partial
4.4	Repeated queries in the median cluster (month)	80.0 %	79.0 %	54.5 %	confirmed
4.4	Fingerprints of long queries (10 min-10 h) that repeat, median cluster	-	82.0 %	87.3 %	confirmed
4.4	Same, for queries under 10 ms	-	32.4 %	49.7 %	confirmed
4.4	Median gap between repeats of a select, median cluster	100 s	42 s	35 s	confirmed

§	Claim	Paper	Provisioned	Serverless	Verdict
4.4	Median gap between repeats, median repeating query	-	599 s	278 s	n/a
4.4	Longest median repeat gap of a cluster	12.0 d	1.0 d	2.7 d	not confirmed
4.6	Selects that repeat a previous select	80.0 %	92.9 %	92.9 %	confirmed
4.6	Selects answered by the real result cache	20.0 %	33.6 %	14.8 %	partial
4.6	Repeating selects that still miss the cache	60.0 %	63.9 %	84.1 %	confirmed
4.6	of those misses: reads system tables	14.0 %	12.4 %	51.2 %	partial
4.6	of those misses: reads external tables	4.0 %	3.3 %	3.7 %	confirmed
4.6	of those misses: a table was written since	54.0 %	24.8 %	5.9 %	partial
3.3	CTAS statements run once in a week	66.0 %	70.8 %	66.3 %	confirmed
3.3	CTAS tables read by exactly one query	80.0 %	8.1 %	13.3 %	not confirmed
3.3	Queries reading a CTAS-created table	40.0 %	17.6 %	2.8 %	not confirmed
4.5	Distinct table sets in a busy cluster-month (10 sampled)	23.5	5,884	-	not confirmed
5.2	Tables with inserts and no deletes the same day	80.0 %	76.9 %	79.1 %	confirmed
5.2	Tables with deletes and no inserts the same day	30.0 %	19.5 %	1.4 %	partial
3.4	Runtime spread, p90 / median (TPC-H 22 queries in DuckDB SF 10: 3.6x)	-	38x	51x	confirmed

3.1 Queries: writes dominate, and the tail is longer

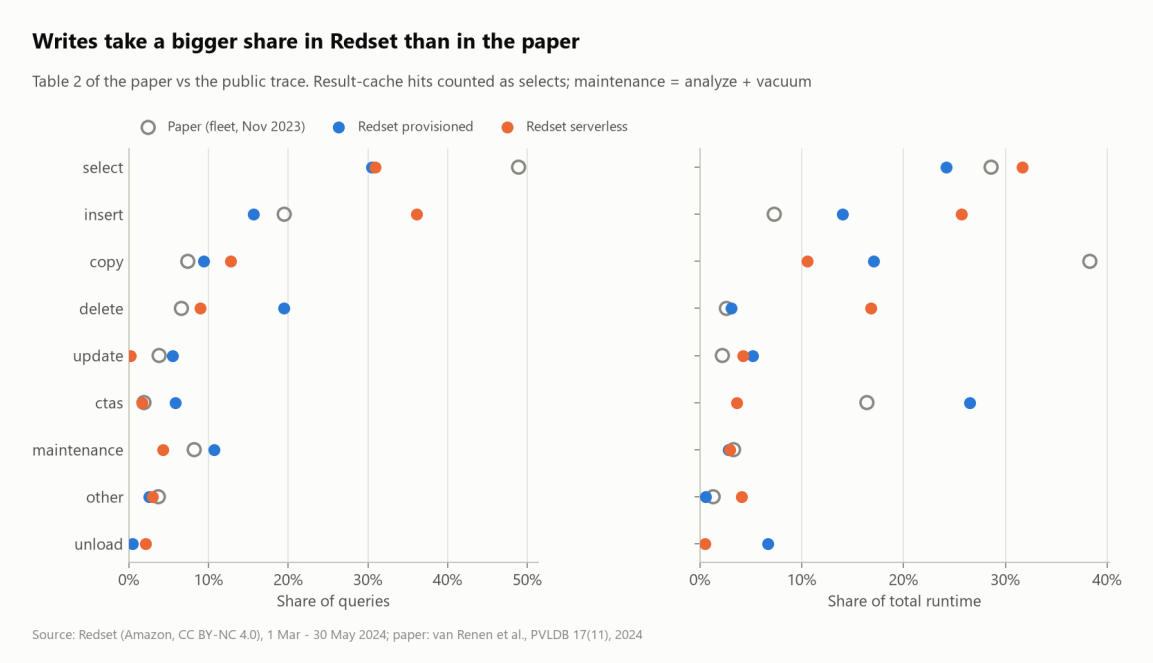


Figure 1: Share of queries and of runtime per query type (the paper's Table 2).

The paper's "analytics = updates" holds strongly (Figure 1). Writes take 65.8 % of provisioned runtime (paper: 66.8 %). Only 30.5 % of queries are selects, against 48.9 % in the paper, so on the public trace writes dominate the query count even more. CTAS statements take 26.5 % of provisioned runtime (paper: 16.4 %) but only 3.6 % on serverless, where plain inserts dominate.

Runtimes are short and heavy-tailed (Figure 2). 62.4 % of provisioned queries finish within 100 ms (paper: 62.0 %), while 0.005 % of queries run longer than an hour and take 12.7 % of the runtime. The paper's Table 3 gives 15.9 % for that bucket; its text says "≈25 %". The ratio of the 99.9th percentile to the median is 7,483 (provisioned) and 7,066 (serverless), against 2,030 in the paper. TPC is narrow by comparison: the 90th percentile of the 22 TPC-H runtimes is 3.6× their median (TPC-DS: 5.0×), against 38× on Redset provisioned. Queries are also simpler than TPC's. An executed Redset query has 1.31 scans and 0.52 joins on average (paper: 2.03 and 0.53), while a TPC-H query in DuckDB has 3.8 scans and 3.5 joins (TPC-DS: 7.4 and 6.8).

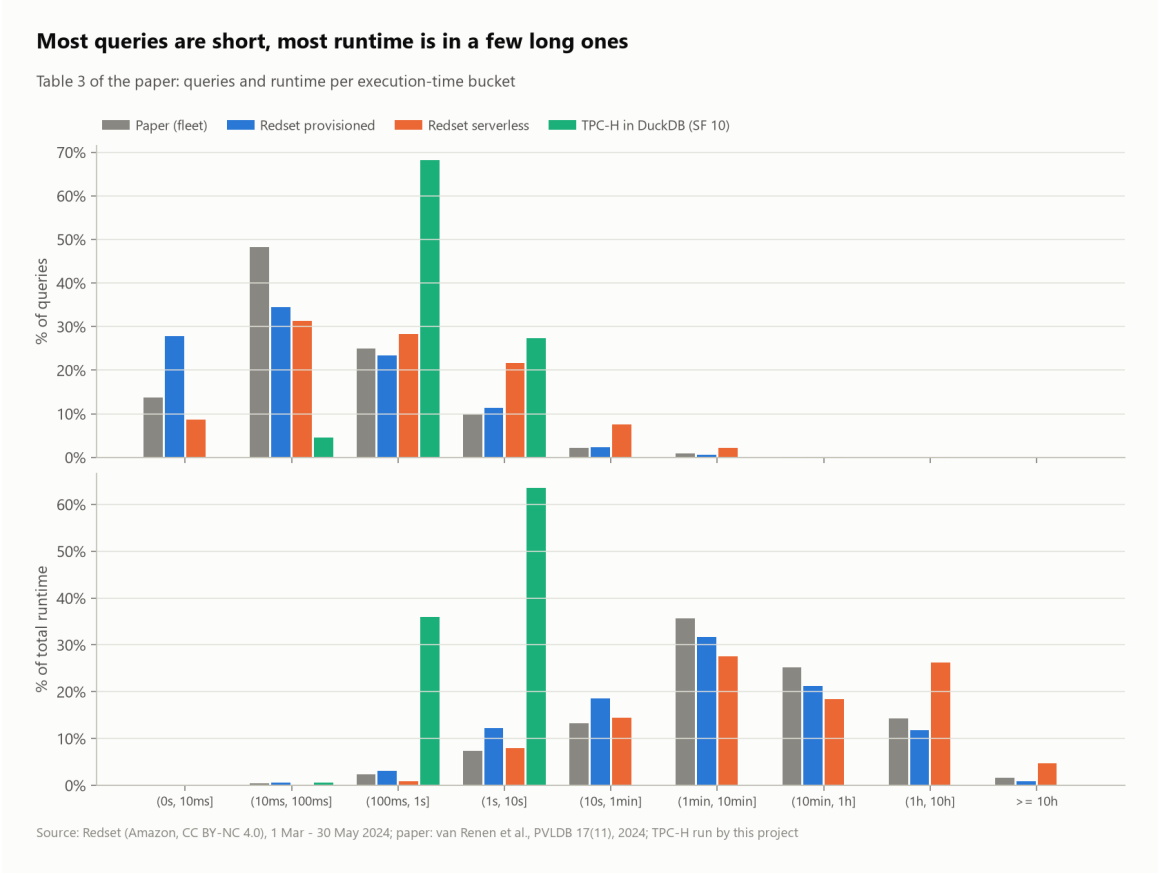


Figure 2: Queries and runtime per execution-time bucket (the paper's Table 3), with TPC-H run in DuckDB.

3.2 Workloads: a stable fleet of unstable clusters

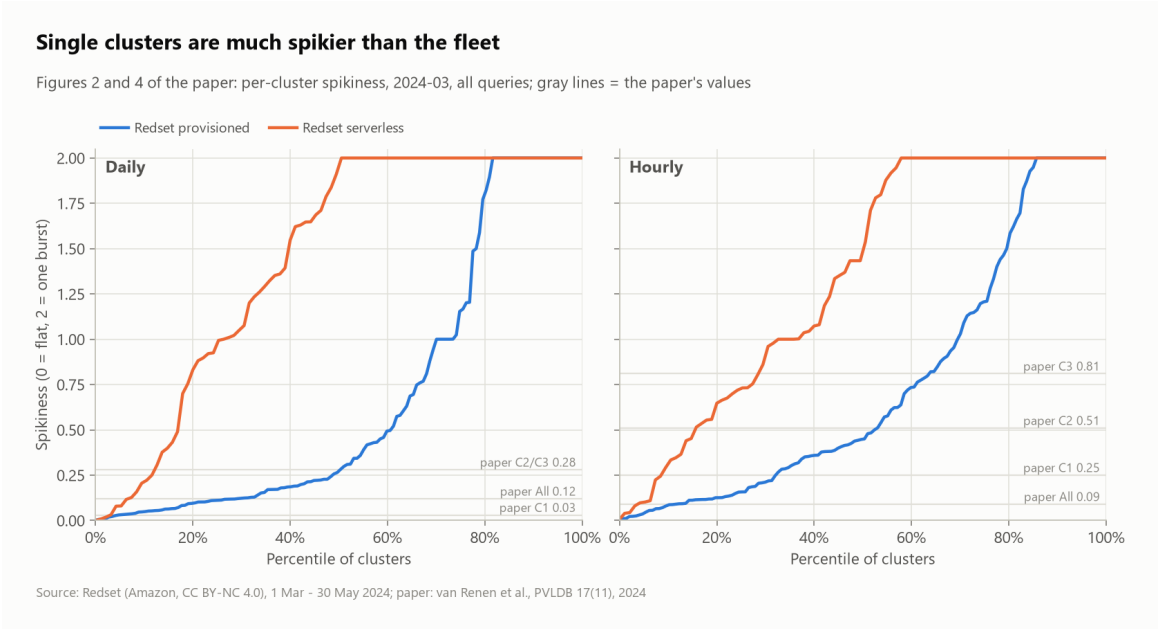


Figure 3: Spikiness per cluster in March 2024 (the paper's Figures 2 and 4).

Aggregated over all clusters, the load is as stable as in the paper (daily spikiness 0.11 vs 0.12). Single clusters are spikier (Figure 3). Half of the provisioned clusters are at most as spiky as the paper's "typical" cluster C2, where the paper reports 75 %. Serverless clusters are far burstier (median daily spikiness 1.96 out of

2): many run a few jobs a day. This is what serverless is for, and it is a workload no TPC run reproduces. Fleet-level stability also varies by month: serverless daily spikiness is 0.15 in March but 0.69 in April.

3.3 Repetition: confirmed, with nuances on timing

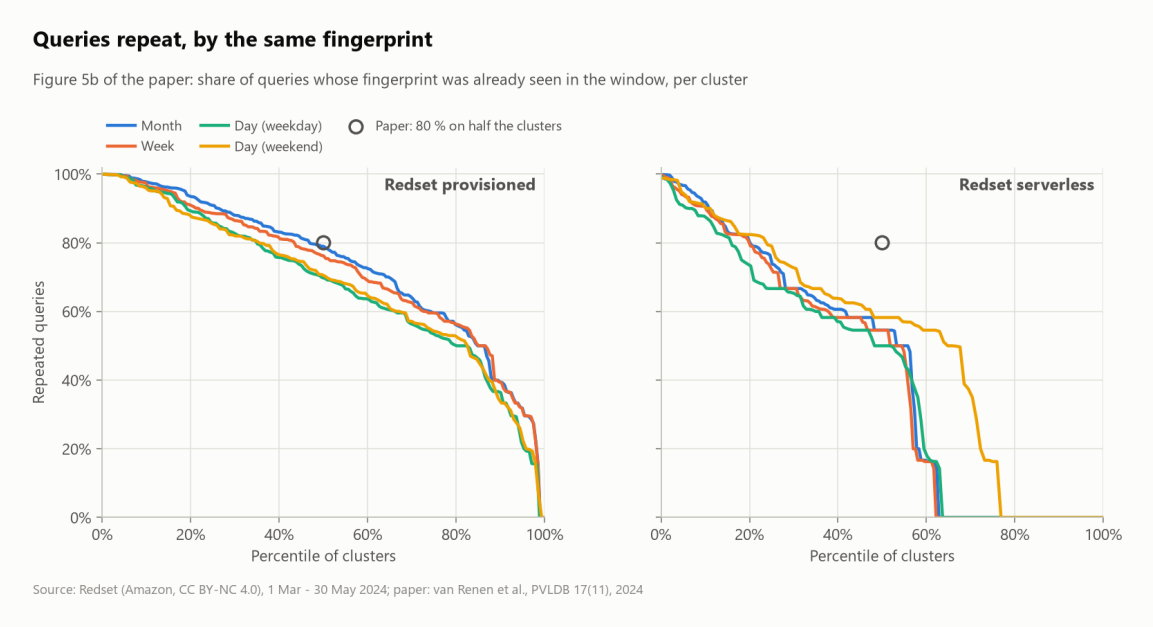


Figure 4: Share of repeated queries per cluster, for several windows (the paper's Figure 5b).

In the median provisioned cluster, 79 % of a month's queries repeat an earlier fingerprint (paper: 80 %; Figure 4). The serverless median is lower (54.5 %), and 37.5 % of serverless clusters show no repetition at all in the month. As in the paper, long queries repeat: in the median cluster, 82 % of the queries whose fingerprint runs 10 min–10 h are repeats, against 32 % under 10 ms. The timing claim depends on how one reads it. Per cluster, the median gap between repeats of a select is 42 s (paper: under 100 s). Per repeating query, the median gap is about ten minutes. The longest per-cluster median is 1.0 day, not 12.

3.4 Tables and CTAS: the clearest disagreements

70.8 % of CTAS statements run once in a given week (paper: 66 %), but their tables are not single-use: only 8.1 % are read by exactly one query (paper: 80 %). 58.9 % are read by 2–5 queries and 24.7 % by none, and 17.6 % of all queries read a CTAS-created table (paper: 40 %). The paper reports 23.5 distinct table sets in busy clusters. Over 10 randomly sampled busy cluster-months, Redset gives a median of 1,438 even after dropping tables read on a single day, and a mean of 97,637 with all tables. Both differences may come from how tables are identified: the paper probably sees stable table names, while Redset ids change with every re-creation. Same-day writes agree: 76.9 % of tables that receive inserts on a day receive no delete that day (paper: 80 %).

4 Extension: what would an ideal result cache save?

The paper reports a cache hit rate of ≈20 %, that ≈80 % of queries repeat, and a breakdown of why repeats miss. Redset allows the counterfactual. I replay every select of every cluster in arrival order. A select is an *ideal hit* if (1) the same fingerprint ran before on the cluster, (2) it reads no system or external table, (3) the previous run had finished when it arrives, and (4) no write to any table it reads committed since the previous run *started*. A write committing during the previous run makes that result stale under snapshot isolation. The last relevant commit is found with a single ASOF join of the per-table read list against the write log, which avoids a quadratic scan. A real hit reports ∼0 ms, so saved runtime uses the median runtime of the fingerprint's real executions. The rules are unit-tested on hand-made traces, including the paper's 3/8 repetition example.

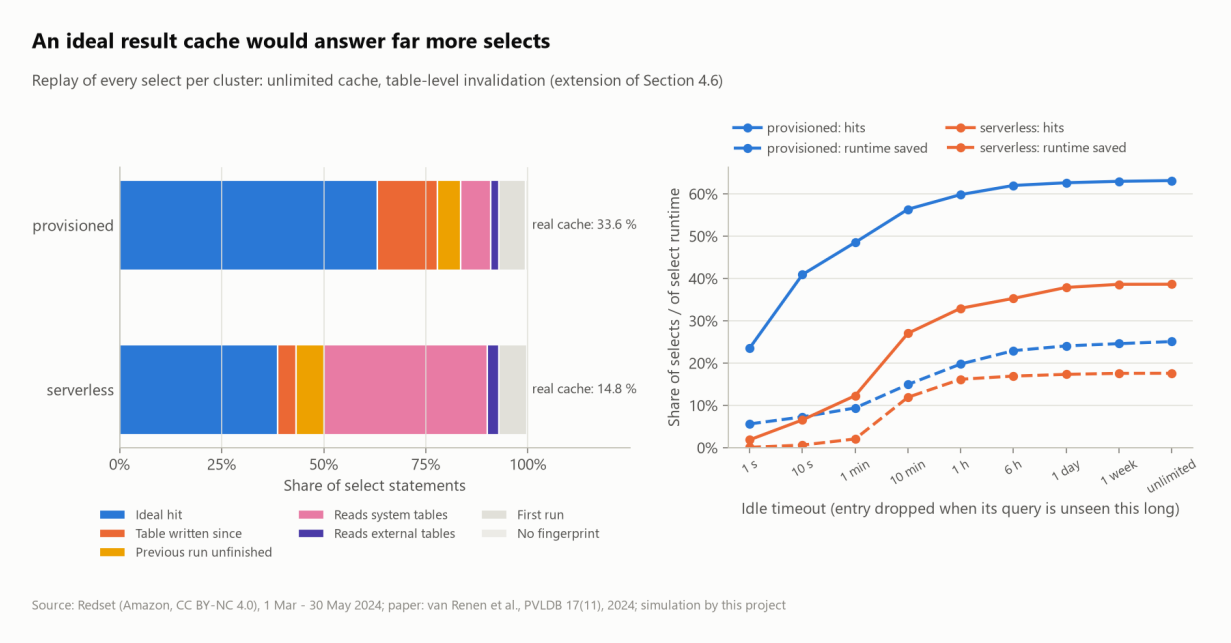


Figure 5: Left: every select classified by the ideal cache. Right: hits and saved runtime under an idle timeout.

Validation. Of the selects Redshift really answered from its cache, the model classifies 97.5 % (provisioned) and 99.1 % (serverless) as ideal hits. The model is therefore not missing a way the real cache works.

Results. An ideal cache would answer 63.1 % of provisioned selects, against the 33.6 % really answered. It would save 25.1 % of select runtime instead of 8.3 %, three times more (Figure 5). On serverless, the figures are 38.6 % vs 14.8 % of selects and 17.6 % vs 1.2 % of runtime. Real misses among repeating selects (63.9 %, paper: ≈60 %) break down as follows. Writes since the previous run explain 24.8 % of them (paper: 54 %), system tables 12.4 % (paper: 14 %) and external tables 3.3 % (paper: 4 %). The remaining ideal hits that Redshift missed are 30.4 % of provisioned selects and 16.9 % of select runtime. They cover causes Redset does not expose: non-deterministic functions, oversized results, eviction and cache scope.

Timeouts. The hits are cheap queries repeated quickly, the savings are expensive queries repeated slowly. An entry dropped after 10 idle minutes keeps 56.3 % of selects as hits (89 % of the ideal) but saves only 14.9 % of runtime (about 60 % of the ideal); one hour gives 59.8 % and 19.8 %. A cache sized by hit rate would therefore miss most of the runtime it could save; retention should be weighted by the cost of the query.

5 Threats to validity

- **Different sample and period.** Redset is a biased sample of clusters, from 2024, while the paper uses November 2023; serverless is a different product. Some disagreements may be real differences between samples rather than errors in either study.
- **Proxies.** The fingerprint is not the query text (it overestimates repetition); table ids are not table names; filters, memory, data types, data skew and storage formats (the paper's Section 5) cannot be measured at all.
- **Interpretation.** The spikiness normalization and the per-cluster reading of the repeat-gap claim are my choices; both are documented and the first is checked against the paper's fleet value.
- **TPC side.** DuckDB on a laptop at SF 10 is not Redshift at SF 3000; only distribution shapes and plan operators are compared.
- **Cache model.** It ignores result size limits, eviction under memory pressure and non-deterministic functions, so it is an upper bound. The 97 % agreement with real hits only shows that it does not *miss* real behavior.

6 Conclusion

On the public trace, the paper's central message holds. Cloud warehouse workloads are write-heavy, repetitive and long-tailed, and TPC captures none of that: TPC-H's runtime spread is ten times narrower and its queries scan three times more tables. Several specific numbers do not transfer: the query mix, CTAS usage and table sets. A benchmark built from "the" fleet numbers would encode one sample, which supports the paper's own call for a configurable benchmark generator. The cache extension turns the paper's qualitative remark into a number: two thirds of the select runtime that an ideal cache could save is not saved today, and recovering it needs cost-aware retention more than a larger hit rate. Everything (SQL, tests, figures and this report) regenerates from the raw trace with one command.

References

[1] A. van Renen, D. Horn, P. Pfeil, K. Vaidya, W. Dong, M. Narayanaswamy, Z. Liu, G. Saxena, A. Kipf, T. Kraska. *Why TPC Is Not Enough: An Analysis of the Amazon Redshift Fleet.* PVLDB 17(11): 3694–3706, 2024. doi:10.14778/3681954.3682031

[2] Amazon. *Redset*, `https://github.com/amazon-science/redset`. CC BY-NC 4.0.

[3] M. Raasveldt, H. Mühleisen. *DuckDB: an Embeddable Analytical Database.* SIGMOD 2019.